Reflexión escrita — TPI Metodología en Sistemas

SaaS odontológico multi-tenant: agenda, pacientes, caja y facturación AFIP/ARCA

Mateo Lautaro Liendo · Metodología en Sistemas · Prof. Juan Cruz Robledo
Universidad Tecnológica Regional Mendoza — Octubre 2026

---

1. De qué trató el proyecto

Este proyecto consistió en el desarrollo inicial de un sistema odontológico bajo la modalidad de *prompting* asistido por inteligencia artificial. A partir de una consigna y de sucesivos prompts fui generando, con el agente, una estructura completa de trabajo: no solo código, sino también los archivos que le indican al agente cómo debe trabajar, como el **AGENTS.md**, la base de conocimiento (**knowledge-base/01–10**), el roadmap (**CHANGES.md**) y el ciclo de vida **OpenSpec/OPSX**. El producto en sí es un SaaS multi-tenant para clínicas odontológicas —gestión de pacientes, agenda de turnos por profesional y box, caja y facturación electrónica—, con backend en FastAPI + PostgreSQL y frontend futuro en React. Mi alcance efectivo fue el **C-01 *foundation-setup***: dejar la fundación corriendo (Docker, SQLAlchemy + Alembic, Pydantic Settings, pytest con 23 tests en verde).

2. Cómo fue el proceso

El trabajo siguió pasos bien marcados. Primero la **fundación**: crear la estructura del proyecto y los archivos necesarios para poder operar. Después vinieron la etapa de **discovery**, la construcción de la **base de conocimiento**, el **roadmap** de 10 changes (C-01 a C-10 en 4 fases) y la implementación con el flujo **propose** → **apply** → **archive**. Cada change exigía leer la parte correspondiente de la KB antes de proponer, lo que al principio me pareció burocrático, pero después entendí su valor: evita que el agente invente reglas de negocio (por ejemplo, RN-01 anti-solapamiento de turnos o RN-03 del CAE previo al cobro). En paralelo aprendí a correr y validar los tests por mi cuenta y a usar a la IA como contraparte para detectar inconsistencias, no como oráculo.

3. Lo que más me costó

Lo más difícil fue la **inestabilidad al usar distintos modelos de razonamiento**. Cambiar de modelo a mitad del trabajo generaba correcciones cruzadas, archivos duplicados o decisiones contradictorias, y tuve que dedicar tiempo a limpiar y uniformar el repositorio. Ligado a eso apareció el problema de las **alucinaciones**: el agente a veces afirmaba cosas que no estaban en la KB ni en el código, y aprendí a frenarlo pidiéndole evidencia (qué archivo, qué línea, qué test lo respalda). Un ejemplo concreto y formativo fue el del **puerto de Docker en 5433 y no en 5432**: el puerto por defecto de PostgreSQL me generaba conflictos con servicios del sistema que estaban fuera de mi alcance corregir. Tras varias pruebas con los tests, mover la base a 5433 eliminó el conflicto. Fue una solución pequeña, pero me enseñó a diagnosticar por descarte en lugar de insistir sobre el mismo error.

4. Lo que más me gustó y lo que investigué

Lo que más me gustó fue entender por dentro el **framework OpenSpec y el agente OpenCode**: ver cómo un prompt bien acotado, con skill y contexto de la KB, produce artefactos trazables (proposal, design, tasks, specs) en vez de código suelto. También disfruté la parte de **testing**: correr *pytest*, ver fallar y pasar, y validar con IA si había inconsistencias me dio una sensación de control que no tenía al inicio. Por mi cuenta investigué por qué se aísla todo por ***clinic_id*** (privacidad y Ley 25.326), por qué las fechas se guardan en **UTC** y se

muestran en hora Argentina, y cómo funcionan la **idempotencia en pagos** y la segregación de capas *api* / *services* / *models* en FastAPI. Esa búsqueda fue lo que más me hizo crecer: pasé de copiar instrucciones a cuestionar decisiones de arquitectura.

5. Cierre

Me llevo dos aprendizajes centrales. Primero, que la IA rinde cuando hay **metodología antes que código**: discovery, KB y roadmap son lo que contiene sus errores. Segundo, que mi rol cambió de programador a **orquestador y verificador**: el que propone, ejecuta tests, detecta alucinaciones y decide. Si volviera a empezar, fijaría un solo modelo desde el inicio y documentaría cada desvío en el momento. El C-01 quedó sellado y archivado; los changes C-02 a C-10 quedan como camino crítico futuro. Más allá del entregable, esta experiencia me mostró cómo se trabaja hoy en sistemas reales: con IA, sí, pero con criterio, pruebas y responsabilidad sobre cada decisión.